Modling script for all data, stores all the fitted paramters

```
% Author: Jiale Jiang
% Editor: Jiale Jiang
```

Load all the needed variables and workspaces

```
load("DataParser.mat", "JettaData", "CamryData", ...
    "JettaMotion", "CamryMotion");
```

Motion fits: acceleration degree 1, velocity degree 2, position degree 3

```
% since there are a lot of obvious signs in our raw data plots that the
% acceleration are actually linear, not constant in our tested courses, so
% I decided to use linear acceleration, quadratic velocity and 3rd order
% position for our model, more details will be shown in our final report;

% Coefficients use elapsed time: tau = time - time_start_sec.
% elapsed time means the actual time that has passed when a motion phase
% start, for example, idle maybe from 0-10 sec, and acceleration starts
% from 10 - 15sec, but when we fit our model, we want to fit it locally, so
% if we are at 12 sec, then the actual acceleration time will be 2 sec,
% this will give us more accurate fit for the model;

% Units: seconds, ft/s^2, ft/s, and distance in ft from the trial start.

% use a for loop
for i = 1:3
    % adding new struct members;
    JettaMotion{i}.fits = struct();
    CamryMotion{i}.fits = struct();

    % Initial idle: zero acceleration/speed and constant position

    % these intervals come from our motiondetection function, it was called
    % in the DataParser.m script file;

    % These following two lines are just used to wrap those intervals so we
    % can use them easier in the vector without having to write a long line
    % just for a vector's interval;
    Jetta_idle1_interval = ...
        JettaMotion{i}.idle_start_i:JettaMotion{i}.idle_end_i;
    JettaMotion{i}.fits.idle1.indices = Jetta_idle1_interval;

    JettaMotion{i}.fits.idle1.time_start_sec = ...
        JettaData{i}.Time_sec(Jetta_idle1_interval(1));
```

```
% compute the actual elapsed time in this motion phase(idle)
tau = JettaData{i}.Time_sec(Jetta_idle1_interval) - ...
    JettaMotion{i}.fits.idle1.time_start_sec;
JettaMotion{i}.fits.idle1.duration_sec = tau(end);

% we use constant position, 0 accel and velocity ,
% during the initial idle of the car, at
% that time, the car didn't start moving yet;
JettaMotion{i}.fits.idle1.fitted_accel_param = 0;

JettaMotion{i}.fits.idle1.fitted_velocity_param = 0;

% this line is to calculate the constant displacement of the car ,
% during its idling
JettaMotion{i}.fits.idle1.fitted_position_param = ...
    JettaData{i}.trip_distanceft(Jetta_idle1_interval(1)) - ...
        JettaData{i}.trip_distanceft(1);

% repeat the same idle modling for Camry
Camry_idle1_interval = ...
    CamryMotion{i}.idle_start_i:CamryMotion{i}.idle_end_i;
CamryMotion{i}.fits.idle1.indices = Camry_idle1_interval;
CamryMotion{i}.fits.idle1.time_start_sec = ...
    CamryData{i}.Time_sec(Camry_idle1_interval(1));
tau = CamryData{i}.Time_sec(Camry_idle1_interval) - ...
    CamryMotion{i}.fits.idle1.time_start_sec;
CamryMotion{i}.fits.idle1.duration_sec = tau(end);
CamryMotion{i}.fits.idle1.fitted_accel_param = 0;
CamryMotion{i}.fits.idle1.fitted_velocity_param = 0;
CamryMotion{i}.fits.idle1.fitted_position_param = ...
    CamryData{i}.trip_distanceft(Camry_idle1_interval(1)) - ...
        CamryData{i}.trip_distanceft(1);

% Acceleration
Jetta_accel_interval = ...
    JettaMotion{i}.accel_start_i:JettaMotion{i}.accel_end_i;

JettaMotion{i}.fits.accel.indices = Jetta_accel_interval;

JettaMotion{i}.fits.accel.time_start_sec = ...
    JettaData{i}.Time_sec(Jetta_accel_interval(1));

tau = JettaData{i}.Time_sec(Jetta_accel_interval) - ...
    JettaMotion{i}.fits.accel.time_start_sec;

JettaMotion{i}.fits.accel.duration_sec = tau(end);

% this line fits the acceleration using first order in the local motion
% phase interval;
JettaMotion{i}.fits.accel.fitted_accel_param = ...
    polyfit(tau, JettaData{i}.filtered_accel(Jetta_accel_interval), 1);

JettaMotion{i}.fits.accel.fitted_velocity_param = ...
    polyint(JettaMotion{i}.fits.accel.fitted_accel_param, ...
```

```
    JettaData{i}.speed_fts(Jetta_accel_interval(1)));

JettaMotion{i}.fits.accel.fitted_position_param = ...
    polyint(JettaMotion{i}.fits.accel.fitted_velocity_param, ...
    JettaData{i}.trip_distanceft(Jetta_accel_interval(1)) - ...
        JettaData{i}.trip_distanceft(1));

Camry_accel_interval = ...
    CamryMotion{i}.accel_start_i:CamryMotion{i}.accel_end_i;

CamryMotion{i}.fits.accel.indices = Camry_accel_interval;

CamryMotion{i}.fits.accel.time_start_sec = ...
    CamryData{i}.Time_sec(Camry_accel_interval(1));

tau = CamryData{i}.Time_sec(Camry_accel_interval) - ...
    CamryMotion{i}.fits.accel.time_start_sec;

CamryMotion{i}.fits.accel.duration_sec = tau(end);

CamryMotion{i}.fits.accel.fitted_accel_param = ...
    polyfit(tau, CamryData{i}.filtered_accel(Camry_accel_interval), 1);

CamryMotion{i}.fits.accel.fitted_velocity_param = ...
    polyint(CamryMotion{i}.fits.accel.fitted_accel_param, ...
    CamryData{i}.speed_fts(Camry_accel_interval(1)));

CamryMotion{i}.fits.accel.fitted_position_param = ...
    polyint(CamryMotion{i}.fits.accel.fitted_velocity_param, ...
    CamryData{i}.trip_distanceft(Camry_accel_interval(1)) - ...
        CamryData{i}.trip_distanceft(1));

% this line first assign the deceleration end point as the end of
% motion point, if the car does not stop and idle after deceleration
Jetta_decel_end = numel(JettaData{i}.Time_sec);

% this if checks if the JettaMotion actually contains a deceleration
% end point, if it does, then it will assign the deceleration end point
% back to the original end point calculated from the motion detection
% code;
% reason for this is that some car does not stop at 0 after decl and
% idle so the motion detection function cannot detect when the car
% stops its deceleration, so then the deceleration end point will just
% set to be the end of the motion;
if isfield(JettaMotion{i}, 'decel_end_i')
    Jetta_decel_end = JettaMotion{i}.decel_end_i;
end

Jetta_decel_interval = JettaMotion{i}.decel_start_i:Jetta_decel_end;

JettaMotion{i}.fits.decel.indices = Jetta_decel_interval;

JettaMotion{i}.fits.decel.time_start_sec = ...
    JettaData{i}.Time_sec(Jetta_decel_interval(1));
```

```
tau = JettaData{i}.Time_sec(Jetta_decel_interval) - ...
    JettaMotion{i}.fits.decel.time_start_sec;

JettaMotion{i}.fits.decel.duration_sec = tau(end);

JettaMotion{i}.fits.decel.fitted_accel_param = ...
    polyfit(tau, JettaData{i}.filtered_accel(Jetta_decel_interval), 1);

JettaMotion{i}.fits.decel.fitted_velocity_param = ...
    polyint(JettaMotion{i}.fits.decel.fitted_accel_param, ...
    JettaData{i}.speed_fts(Jetta_decel_interval(1)));

JettaMotion{i}.fits.decel.fitted_position_param = ...
    polyint(JettaMotion{i}.fits.decel.fitted_velocity_param, ...
    JettaData{i}.trip_distanceft(Jetta_decel_interval(1)) - ...
        JettaData{i}.trip_distanceft(1));

% same logic for Camry
Camry_decel_end = numel(CamryData{i}.Time_sec);
if isfield(CamryMotion{i}, 'decel_end_i')
    Camry_decel_end = CamryMotion{i}.decel_end_i;
end

Camry_decel_interval = CamryMotion{i}.decel_start_i:Camry_decel_end;
CamryMotion{i}.fits.decel.indices = Camry_decel_interval;
CamryMotion{i}.fits.decel.time_start_sec = ...
    CamryData{i}.Time_sec(Camry_decel_interval(1));
tau = CamryData{i}.Time_sec(Camry_decel_interval) - ...
    CamryMotion{i}.fits.decel.time_start_sec;
CamryMotion{i}.fits.decel.duration_sec = tau(end);
CamryMotion{i}.fits.decel.fitted_accel_param = ...
    polyfit(tau, CamryData{i}.filtered_accel(Camry_decel_interval), 1);
CamryMotion{i}.fits.decel.fitted_velocity_param = ...
    polyint(CamryMotion{i}.fits.decel.fitted_accel_param, ...
    CamryData{i}.speed_fts(Camry_decel_interval(1)));
CamryMotion{i}.fits.decel.fitted_position_param = ...
    polyint(CamryMotion{i}.fits.decel.fitted_velocity_param, ...
    CamryData{i}.trip_distanceft(Camry_decel_interval(1)) - ...
        CamryData{i}.trip_distanceft(1));

% this code checks if the motion actually contains a second idle
% section after the deceleration, if so, then it will fit the
% corresponding parameters as well;
if isfield(JettaMotion{i}, 'second_idle_start_i')
    Jetta_idle2_interval = ...
        JettaMotion{i}.second_idle_start_i:...
        JettaMotion{i}.second_idle_end_i;
    JettaMotion{i}.fits.idle2.indices = Jetta_idle2_interval;
    JettaMotion{i}.fits.idle2.time_start_sec = ...
        JettaData{i}.Time_sec(Jetta_idle2_interval(1));
    tau = JettaData{i}.Time_sec(Jetta_idle2_interval) - ...
        JettaMotion{i}.fits.idle2.time_start_sec;
    JettaMotion{i}.fits.idle2.duration_sec = tau(end);
```

```
        JettaMotion{i}.fits.idle2.fitted_accel_param = 0;
        JettaMotion{i}.fits.idle2.fitted_velocity_param = 0;
        JettaMotion{i}.fits.idle2.fitted_position_param = ...
            JettaData{i}.trip_distanceft(Jetta_idle2_interval(1)) - ...
                JettaData{i}.trip_distanceft(1);
    end

    if isfield(CamryMotion{i}, 'second_idle_start_i')
        Camry_idle2_interval = ...
            CamryMotion{i}.second_idle_start_i:...
            CamryMotion{i}.second_idle_end_i;
        CamryMotion{i}.fits.idle2.indices = Camry_idle2_interval;
        CamryMotion{i}.fits.idle2.time_start_sec = ...
            CamryData{i}.Time_sec(Camry_idle2_interval(1));
        tau = CamryData{i}.Time_sec(Camry_idle2_interval) - ...
            CamryMotion{i}.fits.idle2.time_start_sec;
        CamryMotion{i}.fits.idle2.duration_sec = tau(end);
        CamryMotion{i}.fits.idle2.fitted_accel_param = 0;
        CamryMotion{i}.fits.idle2.fitted_velocity_param = 0;
        CamryMotion{i}.fits.idle2.fitted_position_param = ...
            CamryData{i}.trip_distanceft(Camry_idle2_interval(1)) - ...
                CamryData{i}.trip_distanceft(1);
    end

end
```

Save only the updated motion structures;

"-append" will check if the variables already exist in those original files, if so, their saved values will be replaced by new values, if not, then the new variables will be added to the file, this may prevent some accidential value overwrites or sometimes accidential naming inconsistency that results in too many variables in a worksapce;

```
save("DataParser.mat", "JettaMotion", "CamryMotion", "-append");
```

Published with MATLAB® R2024b